MedMind: Medical Website

Software documentation for the Afro-Asian Forum for Innovation and Technology (AFFIAT), Aswan Finals 2026.



TECHNOFuture
KAFR EL SHEIKH



MedMind
Social Health Connection

1. Project Overview | نظرة عامة على المشروع

Field	Value
Project Title	MedMind: Medical Website
Project Type	Web application: a server (Express + SQLite) and plain HTML/CSS/JavaScript pages
Description	MedMind is a medical website that helps you run a simple, regular check on your health with its AI assistant. If you need care, it lists doctors and hospitals you can book. It also explains diseases and their symptoms, so you can tell whether you need to see a doctor, and which kind.

Field	Value
Target Users	Adults over 34 first, then adults 20–30, parents with babies, and teens and young adults. Doctors and hospitals list themselves on it.
Live site	https://zain7856.github.io/MedMind/ (pages only; the chatbot, sign-in and booking need the local server)

Tech Stack | التقنيات المستخدمة

Category	Technology	Purpose
Language	JavaScript	The server and every page
Server	Node.js 22.5 or newer, Express 5.3.0	The API on port 3000: 28 routes in `backend/API/`
Database	SQLite, through `node:sqlite` (built into Node)	Users, symptoms, diseases and appointments in `backend/database/app.db`
Pages	Plain HTML + CSS + JavaScript, no build step	One folder per page in `frontend/Pages/`
AI	Google Gemini (`gemini-2.5-flash`), one `fetch` call	The chat assistant in `backend/Services/Gemini.js`
Images	Web links (Unsplash)	Doctor and hospital photos, saved in `users.Img`

Dependencies | المكتبات

Package	Version	Purpose
express	5.3.0	The web server and its routes. It is the only package in `backend/package.json`.
node:sqlite	Built into Node 22.5+	Talking to the database. Nothing to install.
`process.loadEnvFile`	Built into Node	Reads the Gemini key from `backend/.env`.
`sqlite3` command	System tool	Only for `npm run db:reset`, which builds `app.db` from `app.sql`.

2. Problem Statement | المشكلة

Many people get sick and don't know what to do. They go to the doctor, waste time and money, and still don't know if they are okay.

Why does it matter? | لماذا هذا مهم؟

If people don't see a doctor in time, an illness can become serious, even deadly. MedMind helps people notice early and points them to the right care.

How is it currently solved? | كيف تُحل المشكلة حاليًا؟

Today the only sure way to know is to go to a doctor. People often don't know the symptoms of a disease or which specialty they need, and when they travel they don't know where the nearest doctor or hospital is. MedMind lets them check themselves, read about diseases and book a doctor or hospital.

3. Technical Architecture | البنية التقنية

Project Structure | هيكل المشروع

```
MedMind/
├── backend/
│   ├── server.js              # Set-up: .env, CORS, the 7 route files
│   ├── API/                   # Routes: check input, call a service
│   │   ├── apiUsers.js        # Sign in/up, profile, password, approve
│   │   ├── apiProviders.js    # Doctors, hospitals, one provider
│   │   ├── apiAppointments.js # Book, list bookings, confirm / cancel
│   │   ├── apiDiseases.js     # Diseases: list, one, add, edit, delete
│   │   ├── apiSymptoms.js     # Symptoms and the symptom checker
│   │   ├── apiChat.js         # The AI assistant
│   │   └── apiStats.js        # The three numbers on the home page
│   ├── Services/              # One file per topic, one query per function
│   │   ├── db.js              # Opens database/app.db
│   │   ├── Users.js · Providers.js · Appointments.js
│   │   ├── Diseases.js · Symptoms.js · Stats.js
│   │   └── Gemini.js          # One fetch call to Google Gemini
│   ├── database/
│   │   ├── app.sql            # The 5 tables and all the demo data
│   │   └── app.db             # The SQLite database made from app.sql
│   ├── .env.example           # Shape of .env (GEMINI_API_KEY)
│   └── package.json           # express 5.3.0, npm start / dev / db:reset
├── frontend/
│   ├── index.html             # Goes straight to the home page
│   ├── 404.html
│   ├── Pages/                 # 12 pages: <page>.html, .js and .css
│   │   ├── home/ · about/ · diseases/ · disease/ · symptom-checker/
│   │   ├── doctors/ · hospitals/ · booking/ · chat/
│   │   ├── sign-in/ · sign-up/
│   │   └── profile/           # All 4 roles (+ profile-admin.js)
│   ├── api/                   # One file per topic, one request() each
│   ├── components/            # Header (menu), Footer, helpers.js
│   ├── assets/css/global.css  # Shared colours, spacing and buttons
│   └── imgs/                  # logo.png and the banner
└── docs/                      # This document, poster, slides, diagrams
```

Architecture and Data Flow | تدفق البيانات

```
Browser (frontend/, plain HTML + CSS + JavaScript)
    → a page calls a function in frontend/api/ (for example getDoctors())
        → request("GET", "/doctors") → fetch("http://localhost:3000/doctors")
            → backend/API/apiProviders.js checks the input with plain ifs
                → backend/Services/Providers.js runs one SQL query on app.db
            ← JSON back, or { error: "A friendly message." }
    ← the page draws the answer, or shows the error with alert()

Chat only: backend/API/apiChat.js → Services/Gemini.js → Google Gemini
```

The pages and the server are two separate web servers. The signed-in user is kept in the browser (`localStorage`), and the menu only shows what that user can open.

HOW THE MENU IS BUILT

EVERY PAGE

loadHeader() runs first

components/Header/header.js
draws the same menu on every page.

LOCALSTORAGE

Who is signed in?

getCurrentUser()
null, or { ID, Name, Role, … }

HIDE, DON'T REFUSE

Only usable links show

Chat needs a sign-in. Book now shows for patients only (canBook).

> Chat, booking and profile start with requireSignIn(), which sends a visitor to the sign-in page. Booking also sends anyone who is not a patient back home.

WHO SEES WHICH PAGE

Page	Visitor not signed in	Patient	Doctor / Hospital once approved	Admin
EVERYONE				
🏠 Home	✓	✓	✓	✓
📚 Diseases · one disease	✓	✓	✓	✓
🩺 Symptom checker	✓	✓	✓	✓
👨‍⚕️ Doctors · 🏥 Hospitals	✓	✓	✓	✓
ℹ️ About	✓	✓	✓	✓
VISITORS ONLY				
🔐 Sign in · Sign up	✓	—	—	—
SIGNED IN				
💬 Chat with the AI	—	✓	✓	✓
👤 My profile · password	—	✓	✓	✓
ONE ROLE ONLY				
📅 Book · my bookings	—	✓	—	—
📝 My listing · who booked me	—	—	✓	—
✅ Approve · reject accounts	—	—	—	✓
🗂️ Edit diseases · symptoms	—	—	—	✓

Key Files | الملفات الأساسية

File	Purpose
`backend/server.js`	Reads `.env`, sets the CORS headers so the pages may call the server, and adds the 7 route files.
`backend/API/apiAppointments.js`	Saves a booking only after 4 checks: a date was picked, it is in the future, the person is a Patient, and the doctor or hospital exists and is approved. Every booking starts `Pending`.
`backend/Services/Symptoms.js`	The symptom checker: one SQL query counts, for each disease, how many of the ticked symptoms it has, and puts the highest count first.

File	Purpose
`backend/Services/Gemini.js`	Sends the last 20 chat messages and a system prompt to `gemini-2.5-flash`. The prompt asks for general information only and to urge emergency care for severe symptoms.
`frontend/components/Header/header.js`	Builds the menu on every page. Chat is added only for signed-in users.
`frontend/Pages/profile/profile.js`	One profile page that shows a different section per role: bookings (patient), listing and bookings (doctor / hospital), or the admin tabs.
`backend/database/app.sql`	The 5 tables with `CHECK` rules for roles and statuses, plus the demo data.

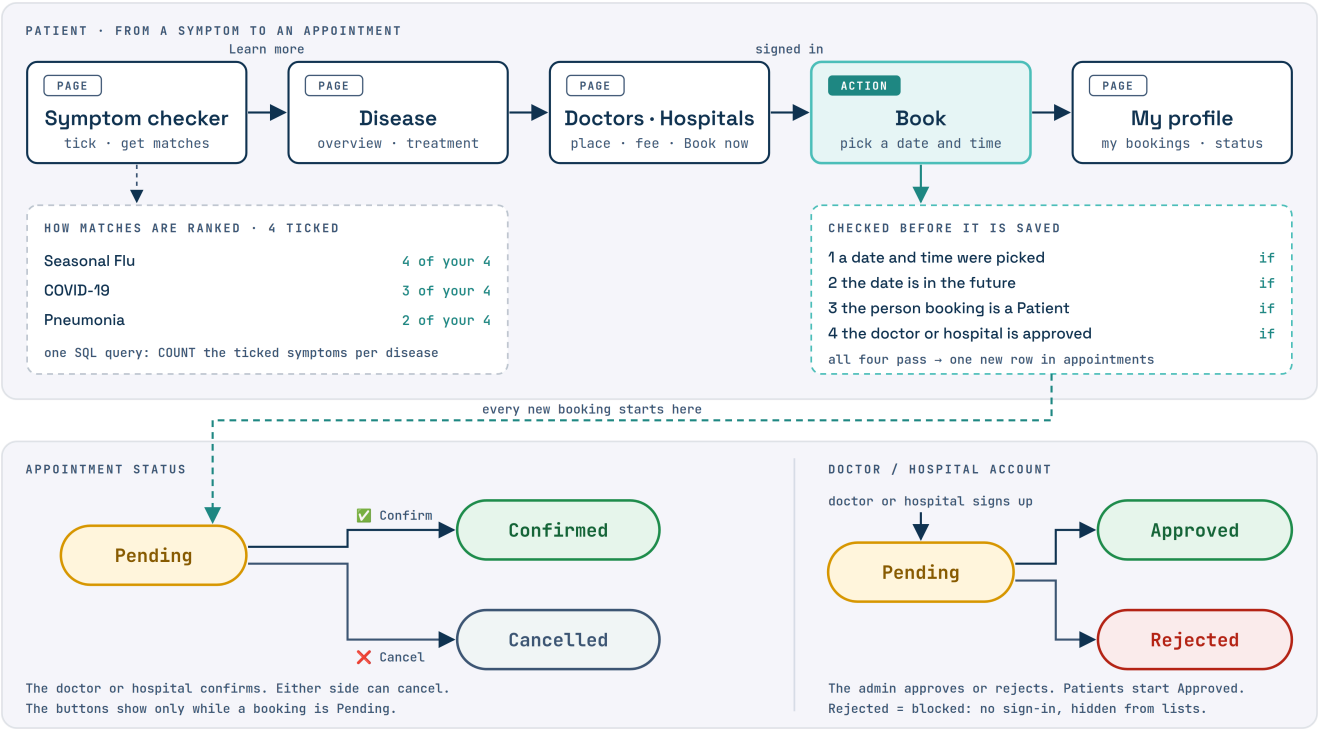
4. Features & User Flow | المميزات ورحلة المستخدم

Feature List | قائمة المميزات

#	Feature	Description	Status
1	Disease library	20 diseases from A to Z. Each page shows an overview, a treatment and its symptoms.	Complete
2	Symptom checker	Tick any of 31 symptoms. Diseases come back ranked by how many of them match. Signed-in users get a button that sends the symptoms to the chat.	Complete
3	Doctors and hospitals	15 doctors in 15 specialties and 10 hospitals, with phone, place, fee (doctors) and services (hospitals). Only approved ones are listed.	Complete
4	Accounts	Sign up as a Patient, Doctor or Hospital. Passwords need at least 6 characters. Doctors and hospitals wait for the admin.	Complete. Passwords are plain text (see Limits).
5	Booking	A patient picks a date and time for a doctor or hospital. It must be in the future.	Complete. Two patients can book the same time.
6	My bookings	Patients see their bookings and can cancel one while it is Pending.	Complete
7	Provider tools	Doctors and hospitals edit their listing, see who booked them, and confirm or cancel.	Complete
8	AI assistant	Signed-in users chat with Gemini. Up to 1000 characters a message; the last 20 messages are sent.	Complete. Needs a working key in `backend/.env`.
9	Admin tools	Approve or reject new doctors and hospitals, reject or delete users, and add, edit or delete diseases and symptoms (and which symptoms each disease has).	Complete

#	Feature	Description	Status
10	Profile	Everyone can edit their profile, change their password, add a profile photo and delete their account.	Complete. The photo stays in this browser only.

User Flow | رحلة المستخدم




```
Visitor:
    Home → Diseases → one disease
         → Symptom checker → tick symptoms → ranked matches → Learn more
         → Doctors / Hospitals → "Sign in to book" → Sign in / Sign up

Patient (signed in):
    Doctors / Hospitals → Book now → Booking (date and time) → Confirm appointment
        → saved as Pending → My profile shows it, with a Cancel button
    Symptom checker → ask the assistant → Chat

Doctor / Hospital (after the admin approves):
    My profile → Appointments with me → Confirm or Cancel each Pending booking

Admin:
    My profile → Pending approvals → Approve / Reject
               → Users → Reject / Delete
               → Diseases / Symptoms → add, edit, delete
```

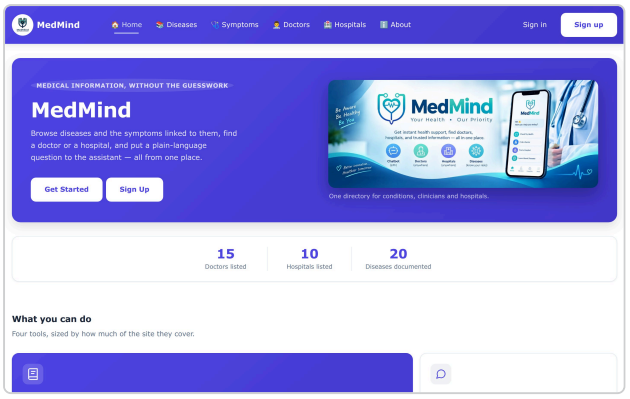
Screen Reference | الشاشات

Screen	File	Who sees it	Description
Home	`Pages/home/`	Everyone	Banner, the live counts (15 doctors, 10 hospitals, 20 diseases), the four main tools and how it works.
About	`Pages/about/`	Everyone	What MedMind is for, and what the logo means.
Diseases	`Pages/diseases/`	Everyone	All diseases as cards, each with "Learn more".
One disease	`Pages/disease/`	Everyone	Overview, treatment, symptoms and a medical disclaimer.
Symptom checker	`Pages/symptom-checker/`	Everyone	31 tick boxes, "Check my symptoms", and the ranked list.
Doctors	`Pages/doctors/`	Everyone	Doctor cards: photo, specialty, phone, place, fee. "Book now" for patients, "Sign in to book" for visitors.
Hospitals	`Pages/hospitals/`	Everyone	Hospital cards: photo, place, services, the same booking button.
Sign in · Sign up	`Pages/sign-in/` · `Pages/sign-up/`	Visitors	Email and password; sign-up adds listing fields for doctors and hospitals.
Chat	`Pages/chat/`	Signed in	The AI assistant.
Booking	`Pages/booking/`	Patients	Name, doctor or hospital, date and time, "Confirm appointment".
My profile	`Pages/profile/`	Signed in	Personal details for everyone, then the section for the user's role.
Not found	`404.html`	Everyone	Shown for a wrong address.

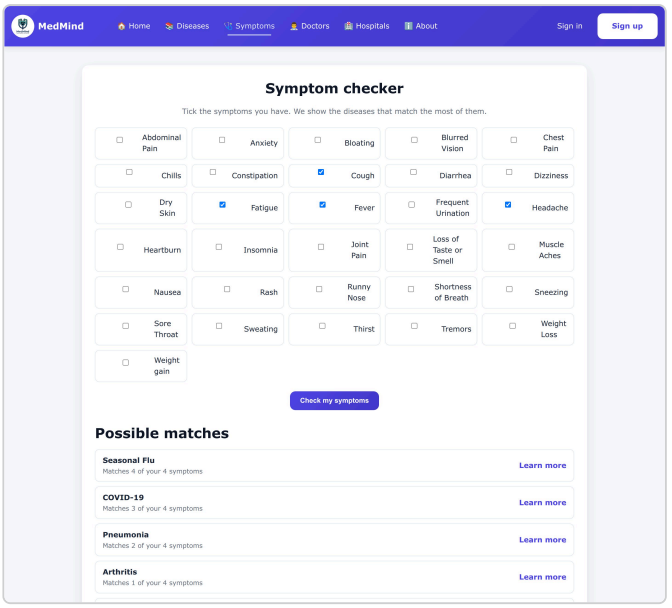
5. UI/UX Design | تصميم واجهة المستخدم

Screenshots | لقطات الشاشة

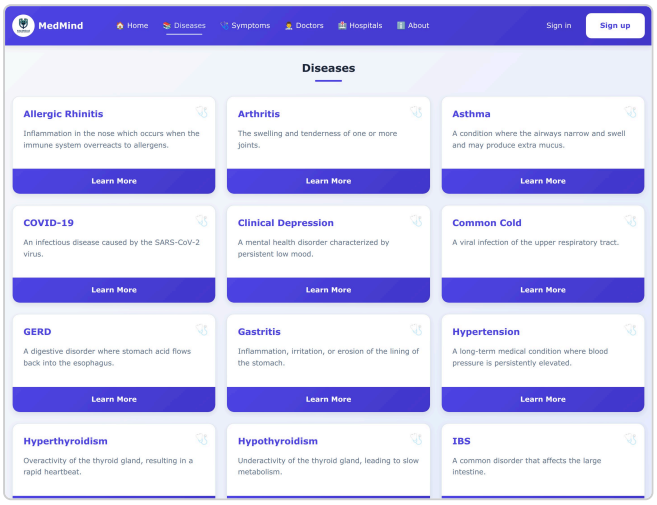
Home



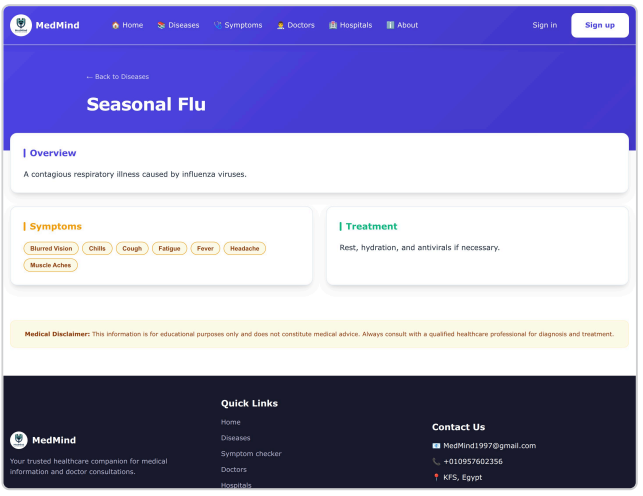
Symptom checker



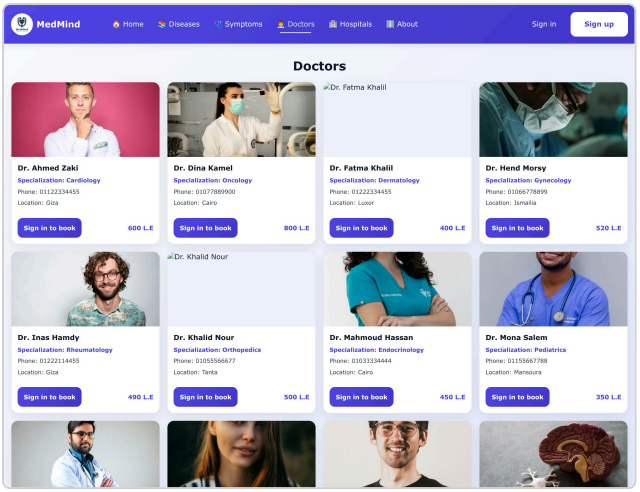
Diseases



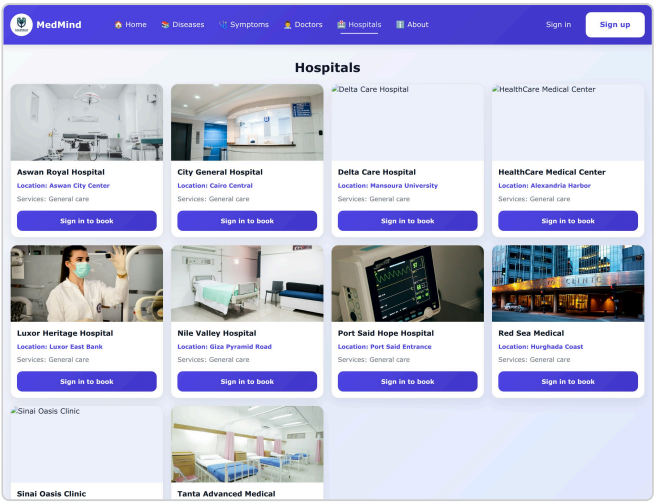
One disease



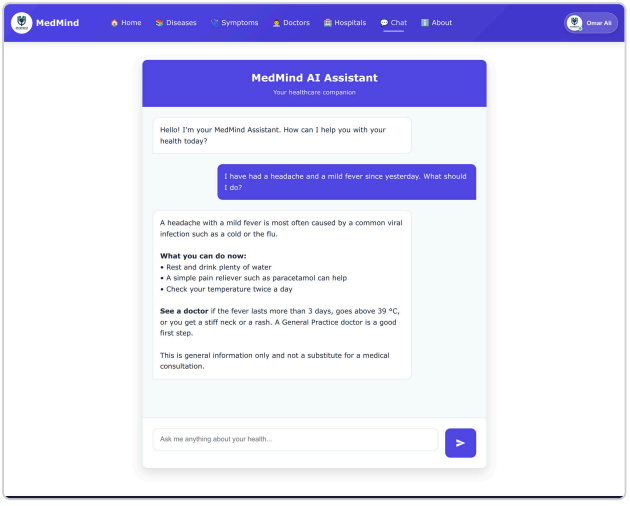
Doctors



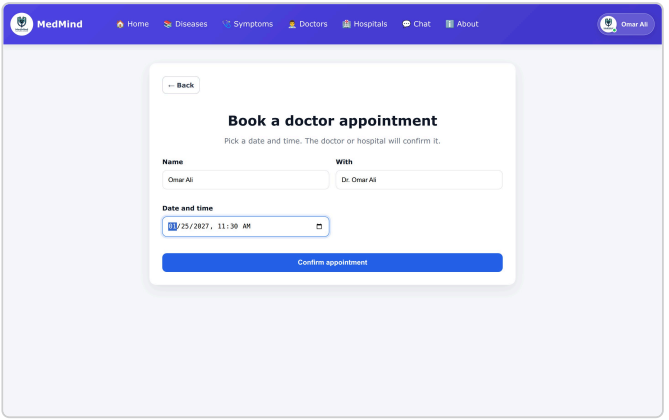
Hospitals



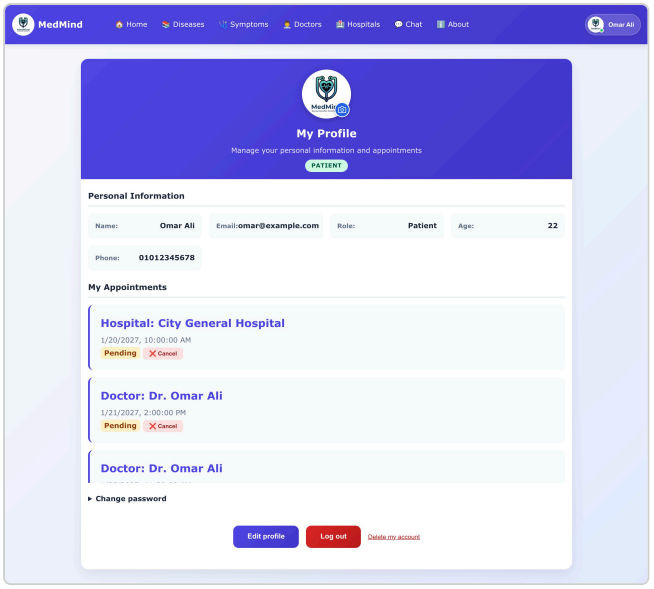
Chat



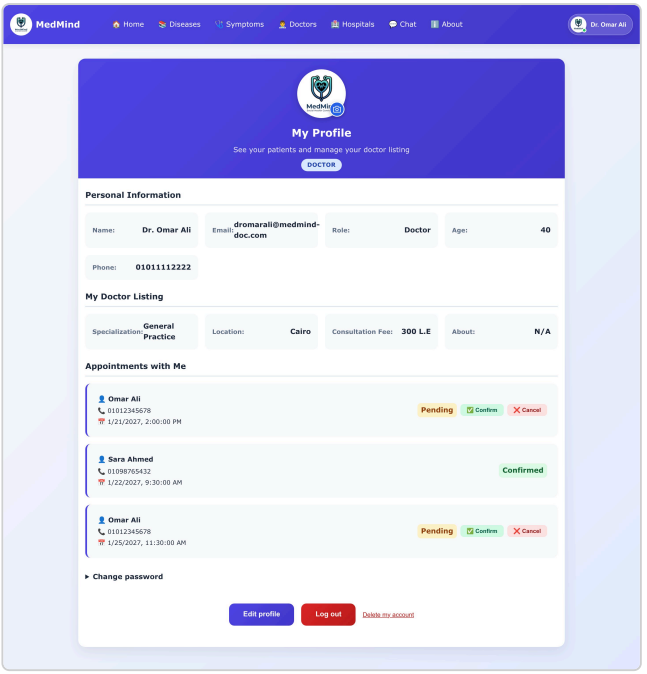
Booking



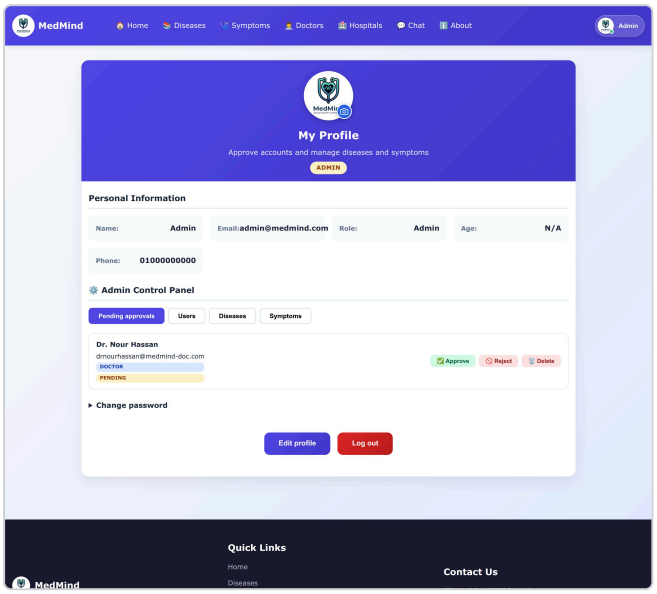
Patient’s profile



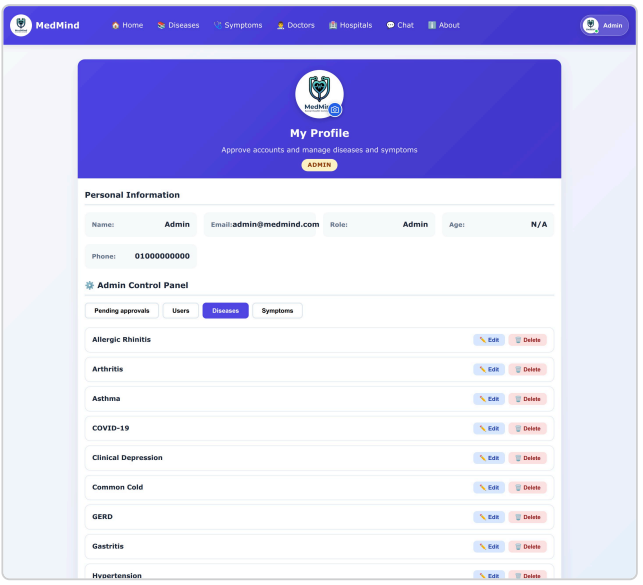
Doctor’s profile

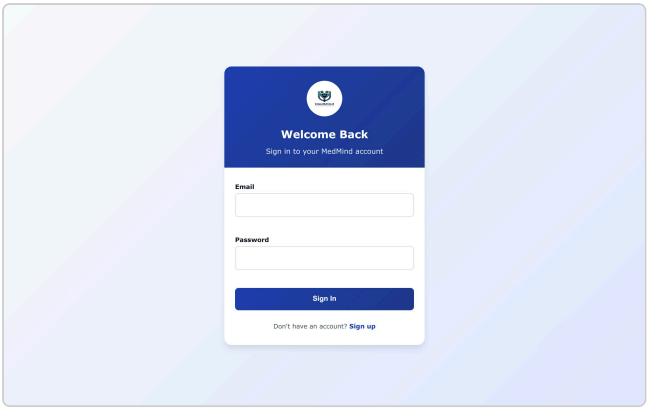
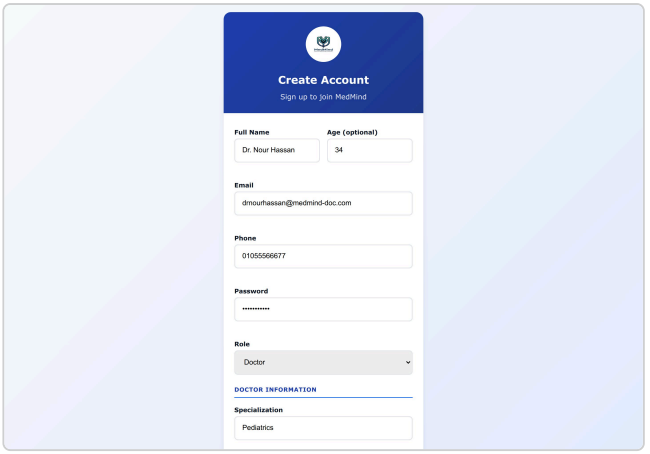


Admin: pending approvals



Admin: diseases



Sign in	Sign up
	

See the pages online: https://zain7856.github.io/MedMind/. The online copy shows the pages. The chatbot, sign-in and booking need the MedMind server, which runs on the laptop at the stand.

Design System | نظام التصميم

Element	Current implementation
Logo	A stethoscope whose tube makes a shield around a heart with a heartbeat line, in a teal-to-blue gradient, above "MedMind" in navy (`frontend/imgs/logo.png`).
Primary color	Indigo `#4f46e5` → `#4338ca` for the header, page banners and main buttons (`components/Header/header.css`)
Secondary color	Green `#059669` (`--secondary-color` in `global.css`)
Text / background	Dark text `#111827` on white and pale slate `#f8fafc`
Status badges	Yellow Pending, green Confirmed / Approved, red Cancel / Reject
Menu	Every item has an icon: 🏠 Home, 📚 Diseases, 🩺 Symptoms, 👨‍⚕️ Doctors, 🏥 Hospitals, 💬 Chat, ℹ️ About
Font	The system font (Segoe UI and similar) in the app. The poster and slides use Lexend, Space Grotesk and JetBrains Mono, with IBM Plex Sans Arabic for the Arabic.
Errors	The server sends `{ error: "A friendly message." }` and the page shows it with `alert`.

Element	Current implementation
Layout	One shared page width (`--page-width: 1500px`) and card grids for lists. The three-line phone menu button is hidden by a CSS bug (ISSUES.md #4).

6. Data Sources | مصادر البيانات

Data Sources | مصادر البيانات

Source	Type	Description
Sign-up form	User input	Name, age, email, phone, password and role go into `users`. Doctors add specialty, place, fee, about and a photo link; hospitals add place, services and a photo link.
Booking form	User input	The date and time go into `appointments`.
Admin forms	User input	New or edited diseases and symptoms, and which symptoms belong to each disease.
Chat box	User input	Sent to Gemini; chats are not saved in the database.
`backend/database/app.sql`	Demo data	34 users (1 admin, 8 patients, 15 doctors, 10 hospitals), 20 diseases, 31 symptoms, 83 symptom–disease links and 4 bookings. The doctors and hospitals are test data made with Claude; the disease information is from the WHO.
Unsplash	Web images	Doctor and hospital photos, linked by URL in `users.Img`.

Database Schema | قاعدة البيانات

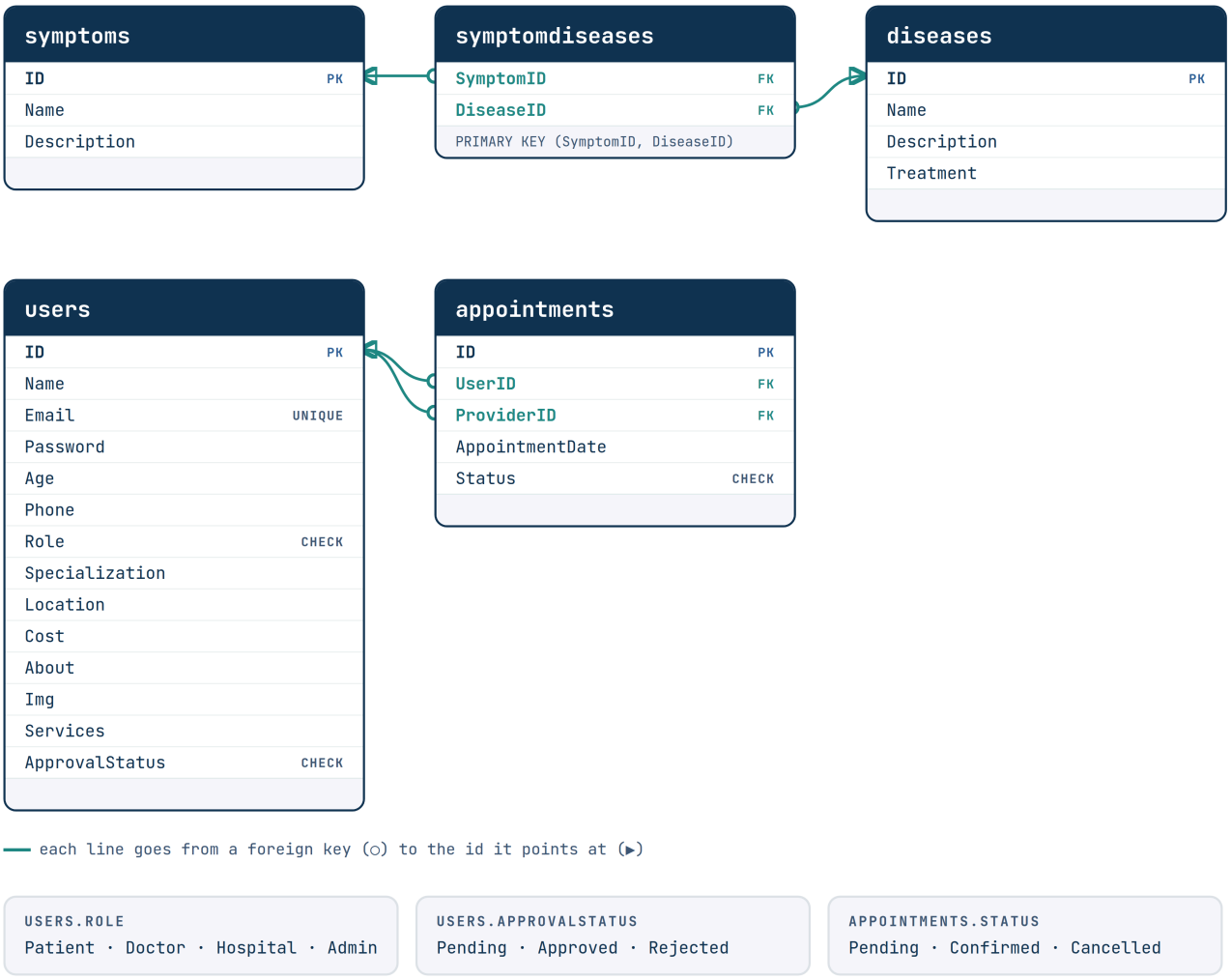


Table	Columns	Description
`users`	`ID`, `Name`, `Email`, `Password`, `Age`, `Phone`, `Role`, `Specialization`, `Location`, `Cost`, `About`, `Img`, `Services`, `ApprovalStatus`	Every account. Doctors and hospitals are users with extra listing columns. `Role` is Patient, Doctor, Hospital or Admin.
`symptoms`	`ID`, `Name`, `Description`	The 31 symptoms.
`diseases`	`ID`, `Name`, `Description`, `Treatment`	The 20 diseases.
`symptomdiseases`	`SymptomID`, `DiseaseID`	One row for each symptom that belongs to a disease.
`appointments`	`ID`, `UserID`, `ProviderID`, `AppointmentDate`, `Status`	Bookings. `ProviderID` is the doctor's or hospital's user ID. `Status` is Pending, Confirmed or Cancelled.

Deleting a user deletes their appointments, and deleting a disease or symptom deletes its links (`ON DELETE CASCADE`).

External APIs | الخدمات الخارجية

API	Purpose	Rate Limit
Google Gemini (`gemini-2.5-flash`)	Answers the chat. The key lives in `backend/.env`.	Google's free-tier limits. MedMind sends at most 1000 characters and the last 20 messages, and does not retry: if Google says no, the user sees "The assistant is busy."

Limits | حدود المشروع

These shortcuts were taken on purpose to keep the code simple. The full list is in the README.

- Passwords are saved as plain text.
- The server does not check who is asking. The pages hide what a user cannot use, but anyone who calls the server directly can do what an admin can. No real patient data.
- The signed-in user is kept only in the browser, and profile photos stay in that browser.
- Booking is first come, first served: two patients can book the same time.
- Rejecting a user blocks them but does not cancel their bookings. The admin cannot be deleted or rejected.
- The AI gives general information, not medical advice, and uses one model with no retry.
- English only.

7. Educational Content | المحتوى التعليمي

Learning Objectives | أهداف التعلم

Through building this project, a student can practice:

1. Build a full-stack web application with Node.js and Express.
2. Design and query a SQLite database.
3. Build sign-in, appointment booking and pages that change with the user's role.
4. Connect Google Gemini to give general medical guidance.
5. Create a web interface with HTML, CSS and JavaScript.

Cultural / Historical Context | السياق الثقافي

Topic	Description
Healthcare access	Helping people get quick medical guidance from home.
Digital health in the region	Connecting patients to local doctors and hospitals.

Curriculum Alignment | الارتباط بالمنهج

Skill Area	What the Student Practices
Problem Solving	Solving a medical-access problem with code.
Computational Thinking	Designing the database tables and the API routes.
Creativity	Designing the pages and the chatbot conversation.
Collaboration	Splitting the code into small modules a team can work on.
Technical Writing	Documenting the API, the database and the features.
Database Design	Designing tables that serve what the app needs.

8. Marketing Plan | خطة تسويقية

Target Audience | الجمهور المستهدف

- **Primary:** adults over 34
- **Secondary:** adults 20–30, and parents with babies
- **Tertiary:** young adults 18–20 and teenagers

Platforms | المنصات

- Web browsers (Chrome, Edge, Brave) on computers, phones and tablets
- The AFFIAT Aswan Finals 2026

Promotion Ideas | أفكار ترويجية

1. Facebook campaigns aimed at the primary audience
2. A weekly email with health and disease-prevention tips
3. Discounts for users who share reviews and experiences
4. A presentation at the AFFIAT Aswan Finals 2026

9. Student Worksheet | ورقة عمل الطالب

My Project Information | معلومات مشروعي

Student Name: Mohamed Ahmed Ali Zain
Project Title: MedMind: Medical Website

What is your project about? | عن ماذا يتحدث مشروعك؟

MedMind helps you check on your health with an AI assistant, regularly, so you feel reassured. If you need care, it shows you doctors and hospitals, and it explains diseases, so you know whether you need to go out to a doctor and which specialty.

What problem does it solve? | ما المشكلة التي يحلها؟

Many people get sick every day and don't know what to do, so they go to the doctor, waste time and money, and still don't know if they are okay.

My Data | بياناتي

Where does data come from?

Two places. The doctors and hospitals are test data made with Claude. The disease information comes from the World Health Organization (WHO). Accounts and bookings are typed in by the people who use the site.

What data is stored?

Users (patients, doctors, hospitals and the admin), diseases, symptoms, which symptoms belong to which disease, and appointments.

What I Learned | ماذا تعلمت

1. **The hardest part:** connecting the frontend to the backend API and managing the database tables.
2. **The most fun part:** connecting the Google Gemini chatbot and watching it answer medical questions.
3. **If I had more time:** live video calls with doctors, and paying for appointments online.
4. **One thing I would do differently:** put an AI assistant across the whole project.

Screenshot of My Project | لقطة من مشروعي

